

ANSIBLE ROLES

Refactoring Monolithic Playbooks
into Reusable, Structured Roles

Professional Reference Documentation

Version 1.0

Table of Contents

1. Introduction
2. The Problem with Monolithic Playbooks
 - 2.1 Symptoms of a Monolith
 - 2.2 Why Refactor
3. What a Role Is
 - 3.1 Definition
 - 3.2 The Standard Directory Layout
 - 3.3 How Ansible Finds Files Inside a Role
4. Refactoring a Monolith into a Role
 - 4.1 The Monolithic Playbook
 - 4.2 Creating the Role Skeleton
 - 4.3 Extracting Tasks
 - 4.4 Extracting Handlers
 - 4.5 Extracting Templates
 - 4.6 Extracting Defaults
 - 4.7 The Resulting Playbook
5. Role Directory Reference
 - 5.1 tasks/
 - 5.2 handlers/
 - 5.3 defaults/ vs. vars/
 - 5.4 templates/ and files/
 - 5.5 meta/
 - 5.6 tests/ and README.md
6. Using Roles in Playbooks
 - 6.1 The roles Keyword
 - 6.2 Passing Variables to Roles
 - 6.3 pre_tasks, roles, and post_tasks
 - 6.4 Conditionals and Tags
7. Role Dependencies
 - 7.1 Declaring Dependencies
 - 7.2 Dependency Execution Order
 - 7.3 Avoiding Duplicate Execution
8. Role Reuse: Galaxy and Collections
9. Best Practices
10. Quick Reference

1. Introduction

A playbook that starts as a few well-organised tasks inevitably grows. New requirements are added, conditions accumulate, templates multiply, and the file becomes difficult to read, reuse, or test. The natural evolution of any serious Ansible project is to refactor that monolithic playbook into one or more **roles**.

A role is a self-contained, reusable unit of automation with a well-defined directory layout. It bundles tasks, handlers, variables, templates, files, and metadata into a single folder that can be included by any playbook — in the same repository or in a completely different project. Roles are the fundamental building block of maintainable Ansible automation.

This document covers:

- The problems that monolithic playbooks create.
- The standard role directory layout and what each subdirectory does.
- A step-by-step refactor from a monolith to a role.
- Role dependencies, variable scoping, and reuse patterns.
- Best practices for writing roles that others can use.

2. The Problem with Monolithic Playbooks

2.1 Symptoms of a Monolith

A playbook has become a monolith when one or more of the following are true:

- A single file exceeds 200–300 lines.
- The same set of tasks appears in more than one playbook.
- Templates are scattered across multiple directories without a clear owner.
- Variables are defined in several places with unclear precedence.
- Copying and pasting a playbook is the fastest way to configure a new server.
- No one is quite sure which tasks are essential and which are historical.
- Conditional logic is deeply nested inside the tasks list.

2.2 Why Refactor

Refactoring into roles addresses every one of these symptoms:

Concern	Monolith	Role
Structure	One large file.	Small, purpose-focused files.
Reuse	Copy and paste.	Include the role by name.
Testability	Test the whole playbook.	Test each role independently.
Variable scoping	Unclear precedence.	defaults/ vs. vars/ is explicit.

Concern	Monolith	Role
Onboarding	Read the entire file.	Read the README and defaults/.
Distribution	Share the whole repository.	Publish to Galaxy or a collection.
Change safety	One change affects everything.	Roles isolate the blast radius.

Table 1 — Monolith vs. role.

3. What a Role Is

3.1 Definition

A role is a directory that follows a specific structure. The structure is a convention, not a configuration: Ansible automatically looks in standard subdirectories for tasks, handlers, variables, and templates, so you do not have to reference them by path inside the role.

Roles are discovered through the configured `roles_path` — by default, a `roles/` directory adjacent to the playbook, and any paths listed under `roles_path` in `ansible.cfg`.

3.2 The Standard Directory Layout

The canonical role layout, as produced by `ansible-galaxy init`:

```
roles/
|-- webserver/
|   |-- README.md           # documentation
|   |-- defaults/
|   |   |-- main.yml        # lowest-precedence defaults
|   |-- files/
|   |   |-- index.html      # static files to copy
|   |-- handlers/
|   |   |-- main.yml        # handlers notified by tasks
|   |-- meta/
|   |   |-- main.yml        # role metadata and dependencies
|   |-- tasks/
|   |   |-- main.yml        # the tasks the role performs
|   |-- templates/
|   |   |-- nginx.conf.j2   # Jinja2 templates
|   |-- tests/
|   |   |-- inventory       # test inventory
|   |   |-- test.yml        # test playbook
|   |-- vars/
|   |   |-- main.yml        # higher-precedence role variables
```

Only `tasks/main.yml` is strictly required for a role to be usable. All other subdirectories are optional; Ansible silently ignores any that are missing.

3.3 How Ansible Finds Files Inside a Role

Inside a role, file and template paths are resolved relative to the role's own subdirectories. This is what makes roles portable.

```
# Inside roles/webserver/tasks/main.yml:

- name: Copy the index page
  ansible.builtin.copy:
    src: index.html          # resolves to roles/webserver/files/index.html
    dest: /var/www/html/index.html

- name: Deploy the nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2       # resolves to roles/webserver/templates/nginx.conf.j2
    dest: /etc/nginx/nginx.conf
```

The same task used outside a role would need `src: files/index.html` and `src: templates/nginx.conf.j2`. Inside a role, the prefixes are implicit.

4. Refactoring a Monolith into a Role

This section walks through a realistic refactor. The starting point is a single playbook that installs and configures nginx on a set of web servers. The end state is a reusable webserver role plus a small playbook that simply includes the role.

4.1 The Monolithic Playbook

```
---
# site.yml -- before refactoring
- name: Configure web servers
  hosts: webserver
  become: true

  vars:
    nginx_port: 80
    nginx_worker_processes: auto
    server_name: example.com
    document_root: /var/www/html

  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present

    - name: Deploy the nginx configuration
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
      notify: Restart nginx

    - name: Deploy the default virtual host
      ansible.builtin.template:
        src: templates/default-site.conf.j2
        dest: /etc/nginx/conf.d/default.conf
        owner: root
        group: root
        mode: '0644'
      notify: Restart nginx
```

```
- name: Copy the index page
  ansible.builtin.copy:
    src: files/index.html
    dest: "{{ document_root }}/index.html"
    owner: www-data
    group: www-data
    mode: '0644'

- name: Ensure nginx is running
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true

handlers:
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

The playbook works. It is also 45 lines, mixes configuration with logic, and cannot be reused by another playbook without copy-paste. Let us refactor.

4.2 Creating the Role Skeleton

The fastest way to create a role is with `ansible-galaxy init`, which generates the full directory layout with placeholder files.

```
# Create a role named "webserver" under roles/
mkdir -p roles
ansible-galaxy init roles/webserver

# Optional: preview what will be created without writing files
ansible-galaxy init --dry-run roles/webserver

# A leaner role with only the directories you will use
ansible-galaxy init --role-skeleton=minimal roles/webserver
```

The result is the standard skeleton described in Section 3.2. Ansible will remove the placeholder files only if you choose to; they are harmless.

4.3 Extracting Tasks

Every task that belonged to the "Configure web servers" play moves into `roles/webserver/tasks/main.yml`. Two adjustments are made during the move:

- File references lose their `files/` and `templates/` prefixes because the role resolves them automatically.
- Hard-coded values become references to role defaults, so the role can be configured by the caller.

```
# roles/webserver/tasks/main.yml
---
- name: Install nginx
  ansible.builtin.package:
    name: "{{ nginx_package }}"
    state: present

- name: Deploy the nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'
  notify: Restart nginx

- name: Deploy the default virtual host
  ansible.builtin.template:
    src: default-site.conf.j2
    dest: /etc/nginx/conf.d/default.conf
    owner: root
    group: root
    mode: '0644'
  notify: Restart nginx

- name: Copy the index page
  ansible.builtin.copy:
    src: index.html
    dest: "{{ document_root }}/index.html"
    owner: "{{ web_owner }}"
    group: "{{ web_group }}"
    mode: '0644'

- name: Ensure nginx is running and enabled
  ansible.builtin.service:
```

```
  name: nginx
  state: started
  enabled: true
```

4.4 Extracting Handlers

The handler moves verbatim into `roles/webserver/handlers/main.yml`. The name must match the `notify` value used in the tasks.

```
# roles/webserver/handlers/main.yml
---
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

4.5 Extracting Templates

The Jinja2 templates move into `roles/webserver/templates/`, keeping their original filenames. Static files move into `roles/webserver/files/`.

```
roles/webserver/  
|-- templates/  
|   |-- nginx.conf.j2  
|   |-- default-site.conf.j2  
|-- files/  
|   |-- index.html
```

Inside the templates, references to variables such as `{{ nginx_port }}` and `{{ server_name }}` continue to work; those variables will now come from the role defaults.

4.6 Extracting Defaults

The `vars:` block from the original playbook becomes the role's `defaults/main.yml`. Defaults have the lowest precedence in Ansible, so a caller can override any of them without editing the role.

```
# roles/webserver/defaults/main.yml  
---  
# Package name -- override for Red Hat family (httpd instead of nginx)  
nginx_package: nginx  
  
# Network configuration  
nginx_port: 80  
nginx_worker_processes: auto  
server_name: example.com  
  
# Content location and ownership  
document_root: /var/www/html  
web_owner: www-data  
web_group: www-data  
  
# Additional configuration directories  
nginx_conf_dir: /etc/nginx  
nginx_sites_dir: /etc/nginx/conf.d
```

Design principle: every value a caller might reasonably want to change should appear in `defaults/main.yml` with a sensible value. Everything that is truly internal to the role — such as the paths to Ansible itself or the version of a supporting library — can live in `vars/main.yml` instead.

4.7 The Resulting Playbook

The playbook is now dramatically shorter. All of the implementation lives in the role; the playbook only says which hosts to configure, which role to use, and how to override its defaults.

```
---  
# site.yml -- after refactoring  
- name: Configure web servers  
  hosts: webserver  
  become: true  
  
  roles:  
    - role: webserver  
      vars:  
        nginx_port: 8080  
        server_name: app.example.com  
        document_root: /var/www/app
```

Notice the two benefits that appear immediately. First, the playbook is readable at a glance — an engineer can see exactly what the play does. Second, the same role can be applied to a second host group with different variables, without any duplication.

```
---
# staging.yml -- reuse the same role with different values
- name: Configure staging web servers
  hosts: staging_webservers
  become: true

  roles:
    - role: webserver
      vars:
        nginx_port: 80
        server_name: staging.example.com
        document_root: /var/www/staging
```

5. Role Directory Reference

5.1 tasks/

The `tasks/` directory holds the work the role performs. `tasks/main.yml` is the entry point and is loaded automatically when the role runs.

Split large task lists into multiple files and include them:

```
# roles/webserver/tasks/main.yml
---
- name: Install and configure nginx
  ansible.builtin.import_tasks: install.yml

- name: Configure virtual hosts
  ansible.builtin.import_tasks: vhosts.yml

- name: Configure TLS certificates
  ansible.builtin.import_tasks: tls.yml
  when: nginx_tls_enabled | bool

# rules:
# - use import_tasks for tasks that are always evaluated at parse time
# - use include_tasks when the file path or condition is dynamic
```

5.2 handlers/

The `handlers/` directory holds handlers that are notified by tasks in the role. They are automatically available to any playbook that uses the role — the caller does not need to define them.

```
# roles/webserver/handlers/main.yml
---
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted

- name: Reload nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded

- name: Validate nginx configuration
  ansible.builtin.command: nginx -t
  changed_when: false
```

5.3 defaults/ vs. vars/

This distinction is the most important concept in role design. Both files define variables; they differ only in precedence.

	defaults/main.yml	vars/main.yml
Precedence	Very low.	Higher than most role-level variables.

	defaults/main.yml	vars/main.yml
Intended use	Values the caller is expected to override.	Constants internal to the role.
Who edits it	The caller, via playbook vars or -e.	The role author.
Examples	Ports, file paths, package names, feature flags.	Platform-specific package names, internal directory names.
Documentation	Listed in the role README.	Rarely documented; treated as implementation detail.
Rule of thumb	Put it here if in doubt.	Use only when overriding would break the role.

Table 2 — defaults/ versus vars/.

Example of the two files working together:

```
# roles/webserver/defaults/main.yml
nginx_port: 80
document_root: /var/www/html
web_owner: www-data
web_group: www-data

# roles/webserver/vars/main.yml
# These names are internal to the role and should not be overridden.
nginx_service_name: nginx
nginx_config_path: /etc/nginx/nginx.conf
nginx_pidfile: /run/nginx.pid
```

5.4 templates/ and files/

Use `templates/` for files that contain Jinja2 expressions and `files/` for static content that should be copied verbatim. Referencing them by name from a task resolves to the correct directory automatically.

```
roles/webserver/
|-- files/
|   |-- index.html
|   |-- robots.txt
|-- templates/
|   |-- nginx.conf.j2
|   |-- default-site.conf.j2

# Reference by name in tasks:
- ansible.builtin.copy:
  src: robots.txt
  dest: "{{ document_root }}/robots.txt"
- ansible.builtin.template:
  src: nginx.conf.j2
  dest: /etc/nginx/nginx.conf
```

5.5 meta/

The `meta/main.yml` file describes the role and declares its dependencies. It is what makes a role distributable through Ansible Galaxy and what allows Ansible to install other roles before running this one.

```
# roles/webserver/meta/main.yml
---
galaxy_info:
  role_name: webserver
  author: Platform Team
  description: Install and configure nginx
  company: Example Corp
  license: MIT
  min_ansible_version: "2.14"
  platforms:
    - name: Debian
      versions: [bookworm, bullseye]
    - name: EL
      versions: ["9", "8"]
  galaxy_tags:
    - nginx
    - webserver

dependencies:
  - role: common
  - role: firewall
  vars:
    firewall_allowed_ports:
      - "{{ nginx_port }}"
```

5.6 tests/ and README.md

These are for the role author. `tests/` typically contains a small playbook and inventory that exercise the role in isolation, useful for CI. `README.md` documents the role for other engineers — what it does, what variables it accepts, and how to use it.

```
# tests/test.yml
---
- hosts: localhost
  become: true
  roles:
    - webserver

# tests/inventory
[all]
localhost ansible_connection=local
```

6. Using Roles in Playbooks

6.1 The roles Keyword

The simplest way to use a role is to list it under the `roles` keyword of a play. The role is included after `pre_tasks` and before tasks.

```
---
- name: Configure the fleet
  hosts: all
  become: true

  roles:
    - common
    - firewall
    - webserver
    - monitoring
```

The older short form `- common` is still valid. The full form `- role: common` is preferred because it allows variables and conditionals to be attached.

6.2 Passing Variables to Roles

There are several ways to pass variables to a role:

Inline with the role entry (highest precedence):

```
roles:
  - role: webserver
    vars:
      nginx_port: 8443
      server_name: api.example.com
```

Through `group_vars` or `host_vars` (recommended for values that vary per host or per group):

```
# group_vars/webserver.yml
nginx_port: 8080
server_name: www.example.com

# The playbook stays simple:
roles:
  - webserver
```

Through play vars (applies to every role in the play):

```
- hosts: webserver
  vars:
    nginx_port: 9090
  roles:
    - webserver
```

6.3 `pre_tasks`, roles, and `post_tasks`

A play executes its sections in a fixed order. Understanding this order is important when a role depends on prior setup or when a role must be followed by verification.

```
- name: Full play structure
  hosts: webservers
  become: true

  pre_tasks:
    - name: Update the package cache
      ansible.builtin.apt:
        update_cache: true
      when: ansible_facts.os_family == "Debian"

    - name: Apply a maintenance notice
      ansible.builtin.copy:
        content: "Maintenance in progress\n"
        dest: /etc/motd

  roles:
    - common
    - webserver

  tasks:
    - name: Run any remaining play-specific tasks
      ansible.builtin.debug:
        msg: "Role execution is complete"

  post_tasks:
    - name: Verify the site is reachable
      ansible.builtin.uri:
        url: "http://localhost:{{ nginx_port }}/health"
        status_code: 200

  handlers:
    - name: Local handler defined in the play
      ansible.builtin.debug:
        msg: "Play-level handler"
```

The execution order is: `pre_tasks`, then `roles`, then `tasks`, then `post_tasks`. Handlers (including those defined by roles) run at the end of the play unless `meta: flush_handlers` is used.

6.4 Conditionals and Tags

A role can be applied conditionally by attaching `when` to its entry. Tags may also be attached so that the role can be selected or skipped with `--tags` and `--skip-tags`.

```
roles:
- role: webserver
  when: "'webserver' in group_names"
  tags: [web, nginx]

- role: database
  when: "'databases' in group_names"
  tags: [db, postgres]

- role: monitoring
  tags: [monitoring]
  when: monitoring_enabled | bool

# Run only the web role:
# ansible-playbook site.yml --tags web
# Skip monitoring:
# ansible-playbook site.yml --skip-tags monitoring
```

7. Role Dependencies

7.1 Declaring Dependencies

A role may declare other roles that must run before it. This is done in `meta/main.yml` under the `dependencies` key.

```
# roles/webserver/meta/main.yml
---
dependencies:
- role: common
- role: firewall
  vars:
    firewall_allowed_ports: [80, 443]
- role: ssl
  when: ssl_enabled | bool
```

7.2 Dependency Execution Order

- Dependencies are executed before the role that declares them.
- If two roles share the same dependency, Ansible detects the duplicate and runs the dependency only once for the play.
- Dependencies may themselves have dependencies, producing a chain executed depth-first.
- Variables declared under a dependency's `vars` key apply only to that dependency, not to the parent role.

7.3 Avoiding Duplicate Execution

If you need a role to run twice within the same play — for example, to configure two different instances — the automatic deduplication can get in the way. There are two approaches.

Approach 1 — use `include_role` with a unique variable scope:

```
- name: Configure the primary instance
  ansible.builtin.include_role:
    name: webserver
  vars:
    instance_name: primary
    nginx_port: 8080

- name: Configure the secondary instance
  ansible.builtin.include_role:
    name: webserver
  vars:
    instance_name: secondary
    nginx_port: 8081
```

Approach 2 — allow_duplicates in meta/main.yml:

```
# roles/webserver/meta/main.yml
---
allow_duplicates: true
```

`include_role` is dynamic and evaluated at runtime; `import_role` is static and resolved at parse time. Both support variables and tags.

8. Role Reuse: Galaxy and Collections

Roles are portable. Once a role has been written and tested, it can be shared in three ways.

Local reuse. Any playbook in the same project can include a role from the project's `roles/` directory. No distribution is required.

Published to Ansible Galaxy. `ansible-galaxy role init` and `ansible-galaxy role import` publish a role to the public Galaxy service. A `requirements.yml` file records which roles a project depends on so that they can be installed deterministically.

```
# requirements.yml
---
roles:
  - name: geerlingguy.nginx
    version: 3.1.4
  - name: geerlingguy.postgresql
    version: 3.4.0

# Install:
# ansible-galaxy role install -r requirements.yml -p roles/
```

Bundled into a collection. Modern Ansible distributes related roles, modules, and plugins together as a **collection**. Collections are the preferred distribution mechanism for anything intended for wide reuse.

```
# requirements.yml -- collection form
---
collections:
  - name: community.general
    version: ">=7.0.0"
  - name: ansible.posix
    version: 1.5.4

# Install:
# ansible-galaxy collection install -r requirements.yml

# Inside a collection, roles live under roles/ and are referenced with the
# fully qualified name:
# roles:
#   - my_namespace.my_collection.webserver
```

9. Best Practices

Design

- **One role, one purpose.** A role called `webserver` should configure a web server, not also the database and the monitoring agent.
- **Put every overridable value in defaults.** If a caller might reasonably want to change it, it belongs in `defaults/main.yml`.
- **Treat vars as private.** Values in `vars/main.yml` should be constants the caller never overrides.
- **Prefix role variables.** A variable called `nginx_port` is safe; a variable called `port` will collide with other roles.
- **Split large task files.** Keep `tasks/main.yml` short and import focused files such as `install.yml` and `configure.yml`.
- **Guard OS-specific tasks with when.** Use `ansible_facts.os_family` so the role supports multiple platforms from a single source.
- **Declare dependencies in meta.** Do not rely on the playbook to order roles correctly; encode the ordering inside the role.

Naming and documentation

- **Name roles with lowercase words.** Use `webserver` or `postgresql`, not `webServer` or `web_server_2`.
- **Write a README for every role.** Document its purpose, its variables, and an example invocation.
- **Populate meta/main.yml.** License, author, minimum Ansible version, and supported platforms all belong there.

Testing and maintenance

- **Test roles in isolation.** The `tests/` directory should contain a small playbook that exercises the role end-to-end.
- **Use molecule for CI.** Molecule drives a role through create, converge, idempotence, and destroy phases in a container or VM.
- **Pin dependencies.** Record exact versions in `requirements.yml` so a run today reproduces a run tomorrow.
- **Version roles.** Bump a version when you change the `defaults` interface or remove variables, so callers can update deliberately.

10. Quick Reference

Role directory layout

```
roles/<role_name>/
|-- tasks/main.yml          # required -- the role's task list
|-- handlers/main.yml       # handlers notified by tasks
|-- defaults/main.yml       # overridable defaults (low precedence)
|-- vars/main.yml           # internal constants (higher precedence)
|-- templates/*.j2          # Jinja2 templates
|-- files/*                  # static files copied verbatim
|-- meta/main.yml           # metadata and dependencies
|-- tests/                  # test playbook and inventory
`-- README.md               # documentation
```

Common commands

Command	Purpose
<code>ansible-galaxy init roles/NAME</code>	Create a role skeleton.
<code>ansible-galaxy init --dry-run roles/NAME</code>	Preview what will be created.
<code>ansible-galaxy role install -r requirements.yml</code>	Install roles listed in a requirements file.
<code>ansible-galaxy collection install -r requirements.yml</code>	Install collections listed in a requirements file.
<code>ansible-galaxy role list</code>	List locally installed roles.
<code>ansible-doc -t role NAME</code>	Show documentation for an installed role.
<code>ansible-playbook site.yml --tags web</code>	Run only the tasks tagged "web".
<code>ansible-playbook site.yml --skip-tags monitoring</code>	Run everything except the "monitoring" tag.
<code>ansible-playbook site.yml --list-tasks</code>	List all tasks, including those from roles.
<code>ansible-playbook site.yml --list-tags</code>	List all tags defined in the playbook and its roles.

Table 3 — Role-related commands.

Variable precedence (role-relevant subset)

Precedence	Source
Lowest	role defaults/
	inventory group_vars/all
	inventory group_vars/parent
	inventory group_vars/child
	inventory host_vars
	play vars
	role vars/
	task vars (including role entry vars)

Precedence	Source
Highest	-e / --extra-vars

Table 4 — Variable precedence relevant to roles.

This document describes Ansible Core behaviour as of version 2.14 and later. Role conventions and the Galaxy / collection ecosystem may evolve between releases; always consult the official documentation for the version in use.